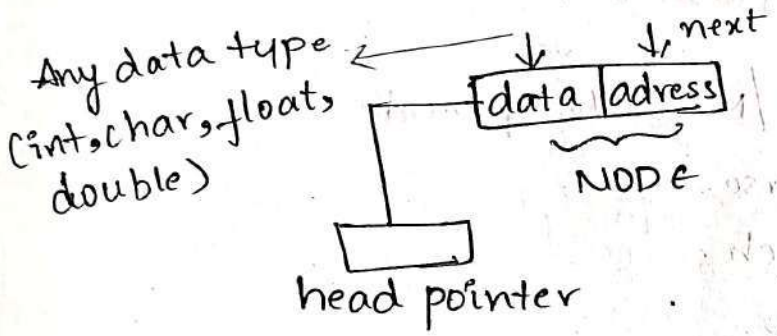
CO-2

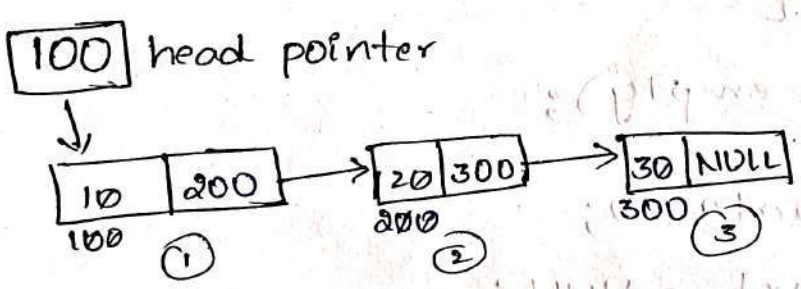
10-01-2025

* Single linked list :

- linked list is consists of two fields one is data field. another field is address field. and it consists of elements known as node
- The elements in the linked list are not arranged in contiguous memory location.
- The elements in linked list are linked using pointer
- They are 4 types of linked list.
  - SLL
  - DLL
  - Circular L.L
  - Doubly Circular LL



- In Single linked list Navigation is done in forward direction only
- It is a linear data structure
- Non-sequential Collection of data items can be stored in
- Every element in the linked list is called as NODE.



Creating a Structure :- (for a node)

Syntax for struct

```
struct * Node {
    int data;
    struct node * next;
};
```

Operations of SLL :-

1. Create
2. Insertion
   - Insert at beginning
   - Insert at End.
   - Insert at position. (specific)
3. Deletion
   - Deletion at begining
   - Deletion at End
   - Deletion at position.
4. Display
5. Node Count
6. Sum of Nodes.
7. Max / Min Ele of node.
8. Reverse.
9. Search.
10. Sort
11. Exit.

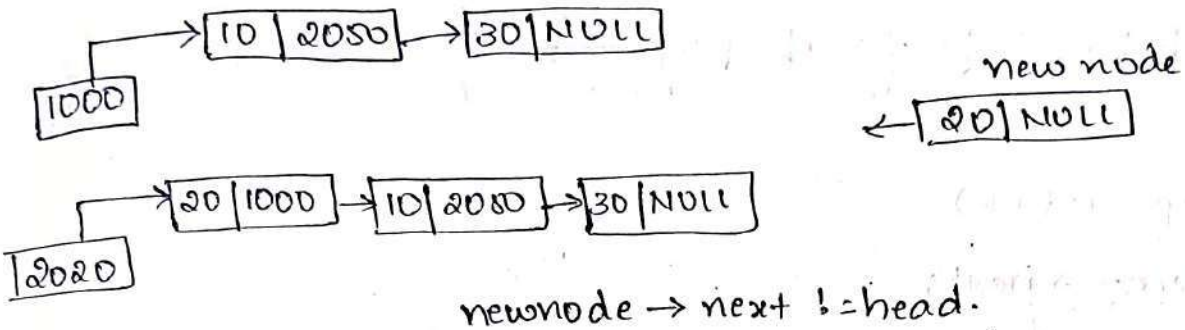
for (Creating Single node)

```
void create ()
{
  int * newnode, *head;
newnode = (struct node*)malloc(size of (struct node));
if (head == NULL) {
    Printf ("List is empty");
}
  new node → data = 10;
  new node → next = NULL;
{
```

```
head = newnode;
}.
```

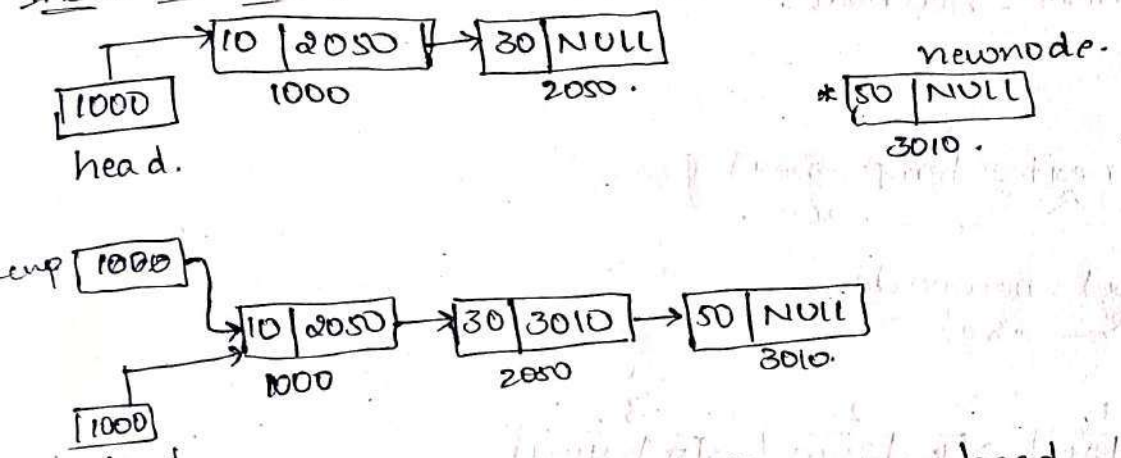
Single linked List:-

Insert at Beginning:-




```
newnode -> next != head.
head = newnode.
```

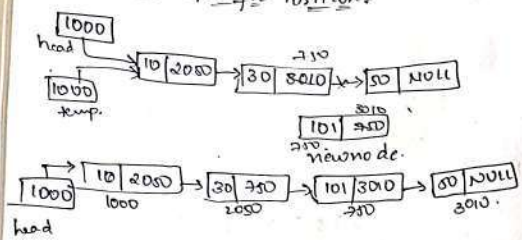
Insert at End:-



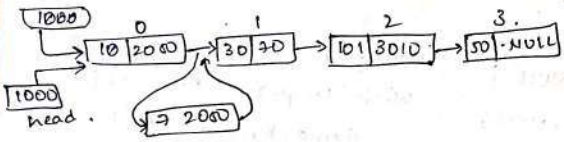

```
While(temp->next != NULL) {
    1000 -> next != NULL;
    2050 != NULL;
    temp = temp -> next
                (2050)
}
temp -> next = newnode.
```

```
=> temp = head.
While (temp -> next != NULL) {
    temp = temp -> next;
}
temp -> next = newnode.
```

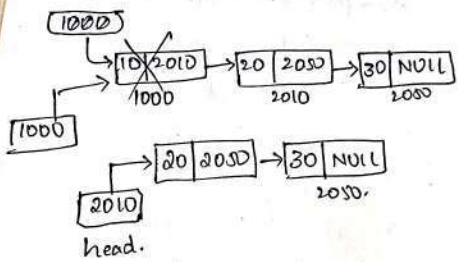
Insert at specific Position:




```
for(i=0;i<pos-1;i++)
{ temp=temp -> next;
}
newnode -> next=temp -> next;
        temp->next = newnode;
i=0; 0<H
  0<0
newnode -> next = temp->next
                  2050.

temp-> next = newnode
          802
```

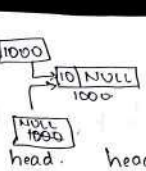


Delete at begining:



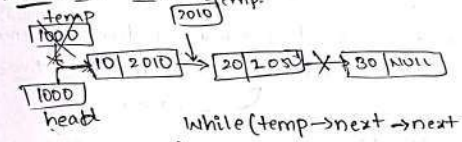
head = temp->next;
2010
free(temp)
1000.

NULL -> list is Empty
head.



head = temp->next.
free(temp).

Delete at End:




```
While(temp->next->next != NULL)
{
  temp=temp->next;
}
  free (temp->next)
  temp->next = NULL
```

tracing:- temp = 1000.

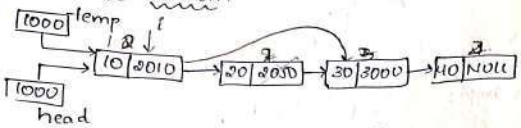
1000 -> next -> next != NULL
2050 != NULL

temp = 2010

2010 -> next -> next != NULL
NULL != NULL X

free.(temp -> next)
temp -> next = NULL
2010 -> next = NULL.

Delete at Position:-




```
temp
int i, pos;
for(i=1; i<pos-1; i++)
{
  temp=temp->next;
}
temp->next=temp->next->next;
```

Tracing

i=1 p<2-1

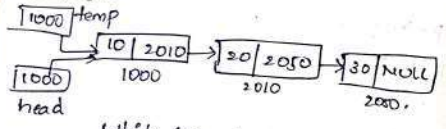
p<1

temp->next=temp->next->next

1000->next = 1000->next->next

2050

Display:-




```
while (temp != NULL)
  printf ("%d", temp->data)
  temp= temp->next.
```

Tracing:-

temp = 1000.

1000 != NULL ✓

Pf ("1000->data"=10)

temp = 2010 != NULL ✓

Pf (" 20");

temp= 2010->next = 2050 != NULL ✓

Pf ("30")

temp= 2050->next = NULL != NULL ✗

No. of Nodes:-

```
{ while (temp != NULL)
    count++;
    temp = temp -> next;
    Pf ("%d", count)
}.
Pf ("%d", count);
```

temp=1000 != NULL ✓

count = 1

temp=2010 != NULL ✓

count = 2

temp= NULL != NULL ✗

count = 3.

Sum of Nodes:-

```
int sum = 0;
while (temp != NULL)
{
  sum=sum+temp -> data;
  temp = temp -> next;
}
Printf ("%d", sum);
```

temp=1000

1000 != NULL

Sum = 0 + 10 = 10

temp=2010

2010 != NULL

Sum = 10+20 = 30

temp=3050

3050 != NULL

Sum = 30+30 = 60

temp=NULL

NULL != NULL ✗

Sum = 60 ✓

Search:-

```
key, flag=0;
while (temp != NULL)
{
  if (temp->data == key)
  {
    flag = 1;
    break;
  }
  temp = temp->next;
}
if (flag == 1)
  Pf ("Ele found");
```

temp=1000 != NULL

10 == 50

temp=2010 != NULL

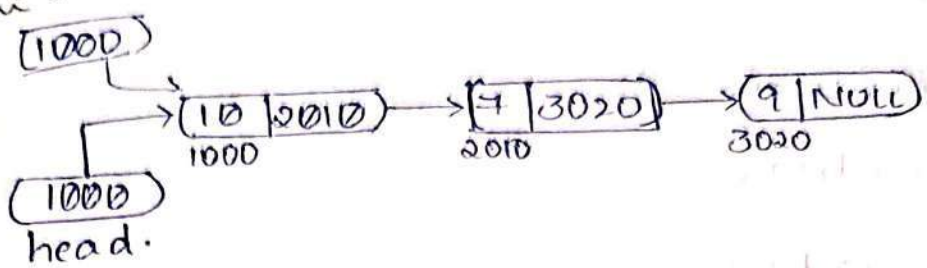
20 == 50

temp=3050 != NULL

30 == 50

temp=NULL != NULL

Sort :-



2 pointers
*p, *q

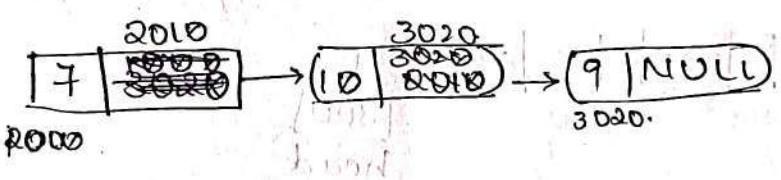
Logic:-

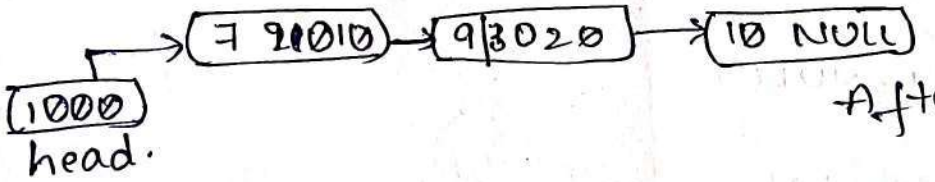
```
for(p=head ; p!=NULL; p=p->next)
{
  for(q=p->next ; q!=NULL; q=q->next)
  {
      if (p->data > q->data)
      {
          temp = p->data;
          p->data = q->data;
          q->data = temp;
      }
  }
}
```

① p = 1000
1000 != NULL ✓
q = 2010
2010 != NULL ✓
10 > 7 ✓
temp = 10
10 = 7 ✓ p
7 = 10 ✓ q

q = q->next
q = 3020
3020 != NULL ✓
9 > 7 ×
q = q->next
NULL != NULL ×

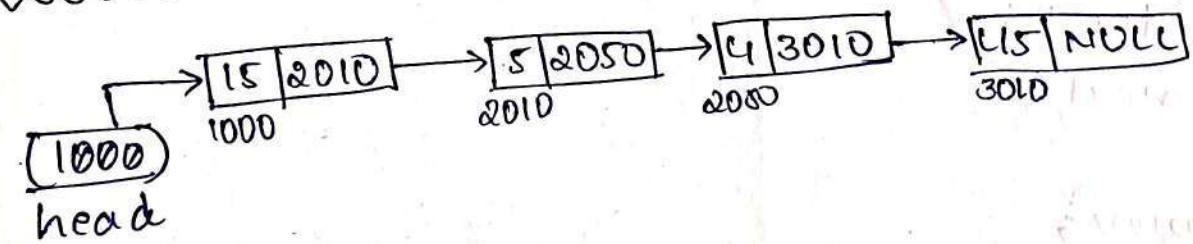
p = p->next
2010 != NULL ✓





After sorting.

'Max-Min( )'



logic:-

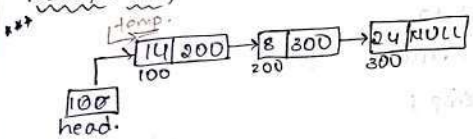
```
max = temp->data;
min = temp->data;
```

```
while (temp!=NULL)
{
  if (max< temp->data)
  {
    max=temp->data;
  }
  if (min >temp->data)
  {
    min=temp->data;
  }
  temp=temp->next;
}
Pf("%d %d", max, min);
```

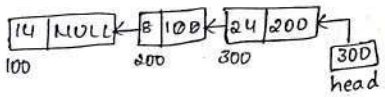
max=15, min=15

15!

Reverse :-( )



After :-



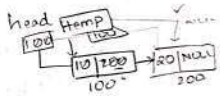
Logic:-

```
While (temp!=NULL)
{
  next=temp->next;
temp->next = prev;
  prev = temp;
  temp = next;
}
head= prev;
```

SINGLE LINKED LIST IMPLEMENTATION:-

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};
struct node *head = NULL;

void insert at beg () {
  struct node *newnode;
  int val;
  newnode = (struct node*)malloc
          (size of (struct node));
  Printf ("Enter value to be inserted
          into the node: ");
  scanf ("%d", &val);
  newnode ->data = val;
  newnode ->next = NULL;
  if (head == NULL) {
    Printf ("Memory allocation failed\n");
    return;
  }
  else {
    newnode ->next = head;
    head = newnode;
  }
}
```

```
void insertatend () {
    struct node *newnode *, temp;
    int val;
    newnode = (struct node*) malloc (size of (struct node));
    Printf ("\n Enter value to be inserted into the node: ");
    Scanf ("%d", &val);
    newnode -> data = val;
    newnode -> next = NULL;
    if (head == NULL) {
        head = newnode;
    }
    else {
        temp = head;
    while (temp -> next != NULL) {
            temp = temp -> next;
    }
    temp -> next = newnode;
    }
}

void insert at spec () {
    struct node *newnode, *temp;
    temp = head;
    int val, i, pos;
    newnode = (struct node *) malloc (size of (struct node));
    Printf ("\n Enter value to be inserted into the node: ");
    scanf ("%d", &val);
    Printf ("\n Enter position after which you want to insert
                a node: ");
    Scanf ("%d", &pos);
    newnode -> data = val;
    newnode -> next = NULL;
    if (pos == 0) {
        newnode -> next = head;
        head = newnode;
    }
    else {
        for (i=0; i < pos-1; i++) {
            temp = temp -> next;
        }
        new node -> next = temp -> next;
        temp -> next = newnode;
        Printf ("Node inserted", pos);
    }
}

void delete at beg ( ) {
    struct node* temp = head;
    if (head == NULL) {
        Printf ("List is Empty");
    }
    else {
        if (temp -> next == NULL) {
            head == NULL;
            free (temp);
        }
        else {
            head = temp -> next;
            free (temp);
        }
    }
}

void delete at end ( ) {
    struct node* temp = head;
    if (head == NULL) {
        Printf ("\n List is Empty");
```



```
if (head -> next == NULL){
    free(head);
    head = NULL;
    return;
}
while (temp -> next -> next != NULL){
    temp = temp -> next;
}
free (temp -> next);
temp -> next = NULL;
Printf ("node deleted");
}

void delete at spe( ){
    Struct node* temp = head;
    int pos, i;
    Printf ("\n Enter the position of the node you want to delete:
    scanf ("%d", & pos);
    if (pos < 1) {
        printf ("Invalid position \n");
        return;
    }
    if (pos == 1) {
        head = head -> next;
        free (temp);
        Printf ("Node at position %d deleted successfully.\n", pos);
        return;
    }
    for (i=1; i< pos-1; i++) {
        temp = temp -> next;
    }
    temp -> next = temp -> next -> next;
    free (temp -> next);
    Printf ("Node at position %d deleted successfully.\n", pos);
}
```

```
void create( ) {
    struct node* newnode, * temp;
    int ele;
    Printf (" Enter data into the node: ");
    scanf ("%d", &ele);
    newnode = (struct node*) malloc (size of (struct node));
    newnode -> data = ele;
    newnode -> next = NULL;
    if (head == NULL) {
    head = newnode;
    }
    else {
        temp = head;
        While (temp -> next != NULL) {
            temp = temp -> next;
        }
        temp -> next = newnode;
    }
}

void display( ){
    struct node* temp;
    temp = head;
    While (temp != NULL) {
        Printf ("%d ", temp -> data);
        temp = temp -> next;
    }
}

void nodecount( ) {
    struct node* temp = head;
    int count = 0;
    While (temp != NULL) {
        count++;
        temp = temp -> next;
    }
    Printf ("\n No. of nodes = %d", count);
}
```

```
void search() {
  struct node* temp = head;
  int ele, flgag=0;
  Printf("\n Enter element to be searched: ");
  scanf("%d", &ele);
  while (temp != NULL) {
    if (temp -> data == ele) {
      flag = 1;
      break;
    }
    temp = temp -> next;
  }
  ~~temp~~ if (flag == 1)
        printf("Element found");
      else
        printf("Element not found");
  }

void reverse() {
  struct node* prev = NULL, *temp=head, *next = NULL;
  while (temp != NULL) {
    next = temp -> next;
    temp -> next = prev;
    prev = temp;
    temp = next;
  }
  head = prev;
  Printf("\n List Reversed Successfully.\n");
}
```

```
void sort() {
  struct node *p, *q;
  int t;
  for (p=head; p!=NULL; p=p->next) {
    for (q=p->next; q!=NULL; q=q->next) {
      if (p->data > q->data) {
        t = p->data;
        p->data = q->data;
        q->data = t;
      }
    }
  }

  void node Sum() {
    struct node* temp=head;
    int sum=0;
    while(temp!=NULL) {
      sum=sum+temp->data;
      temp=temp->next;
    }
    Printf("\n Sum of node elements = %d", sum);
  }

  void max-min ele() {
    struct node* temp = head;
    int max, min;
    if (temp == NULL) {
      Printf("\n List is Empty");
      return;
    }
    max = temp -> data;
    min = temp -> data;
```

```
while (temp != NULL) {
    if (temp -> data > max)
        max = temp -> data;
    if (temp -> data < min)
        min = temp -> data;
    temp = temp -> next;
}
Printf("\n Maximum element = %d. Min element = %d",
            max, min);
}
int main (){
  int i, n, opt;
  while (1) {
    Printf ("\n 1. Create \n 2. Insert \n. 3. Insert at end \n.
      4. Insert at specific position \n. 5. Deletetion at begin
      6. Deletetion at end \n. 7. Deleletion at spec. pos. \n.
      8. Display \n. 9. Node count \n. 10. search \n.
      11. Reverse \n. 12. sort \n. 13. Exit \n. 14. Sum \n.
      15. Max-Min element ");
    Printf (" \n Enter your option: ");
    scanf ("%d", & opt");
    switch (opt) {
      case 1:
        Printf ("\n Enter no. of nodes you want to create:
        scanf ("%d", &n);
        for (i=1; i<=n; i++) {
            create ();
        }
        break;
      case 2:
        insert at beg ();
        break;
      case 3:
        insert at end ();
        break;
      case 4:
        insert at spec ();
        break;
      case 5:
        delete at beg ();
        break;
      case 6:
        delete at end ();
        break;
      case 7:
        delete at spec ();
        break;
      case 8:
        Printf ("\n The elements in the linked list are: \n");
        display ();
        break;
      case 9:
        nodecount ();
        break;
      case 10:
        search ();
        break;
      case 11:
        reverse ();
        break;
      case 12:
        sort ();
      case 13:
        exit (0);
        break;
      case 14: nodesum()
        ~~max-min-ele(-)~~;
        break;
      case 15:
        max-min ele ();
        break;
      default:
        Pf ("\n Invalid option. pls try
            again: ");
    }
    return 0;
```

Stack Using Linked List:-

- The major problem with the stack implimentation using an array it works only for a fixed number of data values. that means the amount of data must be specified at the beginning of the implimentation itself.
- The stack implimentation using Linked list can work for an unlimited number of values. that means stack implimentation using linked list works for the variable size of data. So there is no need to fix the size at the beginning of the implimentation. And it organise as many data values as keep on.
- Steps followed by stacks

→ Every New element is inserted as top element that means every newly inserted element is pointed by top

→ If we want to remove an element from the stack simply remove the node which is pointed by top & moving top to the previous node & the next field of first element must be always NULL

Operations of Stack:-

(1) Push → insert

(2) Pop → delete.

(3) display → Print

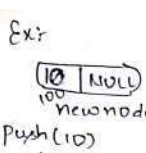
(4) Peek. → top.

Principle/Rule:- followed by stack is LIFO

"last in first out".

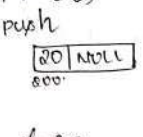
Ex:

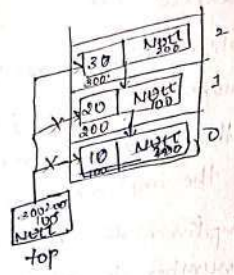
N=3



Push(10)

push





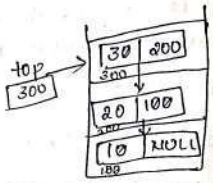
Logic:-

newnode→next=top

top=newnode.

(push operation)

Ex:- pop operation.



logic:-

```
pop()
{
  temp=top;
  top=top→next;
  free (temp)
  Printf ("%d", temp→data);
  free (temp);
}
```

tracing

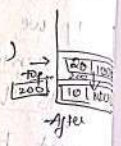
temp=300.

top=200

Printf ("%d", 30)

free (30);

}.



after

Ex:- Peek Operation.

```
peek()
{
Printf ("%d", top→data); // 20
}.
```

```
100 != NULL
{
Pf("10");
temp=NULL;
}
```

```
temp=200
while (200 != NULL)
{
Pf("%d", 20);
temp=100 }
```

Ex:- Display()

```
{
  temp=top;
While(temp != NULL)
{
Printf ("%d", temp→data
  temp=temp→next;
}
```

Program:-

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
//Node structure
Struct Node {
    int data;
    Struct node* next;
};
// Global pointer to the top of the stack
Struct Node *top = NULL;
void push();
int pop();
int peek();
int isEmpty();
int void display();
int main()
{
int choice, value;
while (1) {
  Printf ("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
Printf ("Enter your choice: ");
Scanf ("%d", &choice);
switch (choice){
  case 1:-
      push();
      break;
  case 2:-
      value= POP();
      if (value != 0) {
      Printf ("Popped value: %d\n", value);
      }
      break;
  case 3:-
      value=peek();
      if (value != 0) {
      Printf ("Top element : %d\n", value);
      }break;
  case 4:
      display();
      break;
  case 5:-
      Printf ("Existing...\n");
      exit(0);
  default:
      Printf ("Invalid Choice !\n");
  }
}
return 0;
}
// Function to push an element onto stack
void push() {
int value;
Printf ("Enter value to push: ");
scanf ("%d", &value);
struct Node* newNode = (struct Node*)malloc (size of (struct Node
newNode → data = value;
newNode → next = top;
top = newNode;
```

```
Printf("%d pushed to stack\n",value);
}
//pop function.
int pop() {
    if (isEmpty()) {
        Pf("stack Underflow\n");
        return -1;
    }
    Choice
struct Node* temp = top;
int value = temp->data;
top = top->next;
free(temp);
return value;
}
//Peek
int peek() {
    if(isEmpty()) {
        Pf("stack is empty\n");
        return -1;
    }
    return top->data;
}
// Check stack is empty.
int isEmpty() {
    return top==NULL;
}
//display.
void display() {
    if (isEmpty()) {
    Pf("stack is Empty\n");
    }
struct Node* temp=top;
Pf("stack elements:");
while (temp != NULL) {
    Pf("%d", temp->data);
    temp = temp->next;
}
Printf("\n");
}.
```

Input & output:-

1. Push 2. Pop 3. Peek
4. Display 5. Exit

① Enter choice: 3.
stack is Empty
return -1

② Enter choice:-1
Enter value to push: 11
11 pushed to stack.

③ Enter choice: 2
pop (11)

④ Enter choice:-1
Enter value to push: 87
87 pushed to stack

⑤ Enter choice: 1
Enter value to push: 5
5 push.

⑥ display:-
87 5 5 87.

Queue Using Linked List:-

- Queue is a linear data structure. It follows first in first out principle. Qu
- Queue Support Operations like Enque & Deque.

Rule | Principle
FIFO [First in First Out]

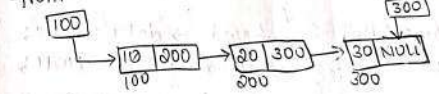
it can be implimented using arrays and also using linked list. The Benefits of implimenting Queue is in Linked list is memory can be allocated dynamically.

Operations of Queue:-

1. Insertion → [Enqueue / front]: insert operation in linked list of an element is added from end of the queue. The new element which is added becomes the last element of the Queue.
2. Deletion → [Dequeue / rear]

Deletion in a linked list means removes the element which was first inserted in the Queue. that is always the 1st element in the Queue will be removed.

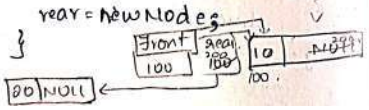
- Here we use 2 pointers that are front & rear. front is used to remove the elements & rear is used to insert the elements.



Program:-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
//Global variable.
struct Node* front = NULL;
struct Node* rear = NULL;
//Function Prototype.
void enqueue(int value);
int dequeue();
int peek();
void display();
```

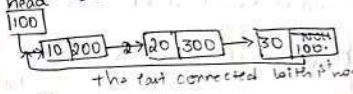
```
int main ()
{
  int choice, value;
  while (1)
  {
    Printf ("\nQueue Operations: \n");
    Printf ("1. Enqueue\n");
    Printf ("2. Dequeue\n");
    Printf ("3. Peek \n");
    Printf ("4. Display\n");
    Printf ("5. Exit \n");
    Printf ("Enter your choice : ");
    Scanf ("%d", & choice);
  switch (choice) {
    case 1:
      Pf ("Enter value to enqueue: ");
      Scanf ("%d", &value);
      enqueue (value);
      break;
    case 2:
      value = dequeue ();
      if (value ! =-1)
      {
        Pf ("Dequeued value: %d\n",
                value);
      }
      else{
        Pf ("Queue underflow! \n");
      }
      break;
    Case 3:
      Value = prek ();
      if (value ! = -1)
      {
        Pf ("Front element : %d\n",
                value);
      }
      else {
        Pf ("Queue is empty! \n");
      }
      break;
    case 4:
      display ();
      break;
    case 5:
      Printf ("Exiting... \n");
      exit (0);
    default:
      Printf ("Invalid choice \n");
    }
  }
  return 0;
}

void enqueue (int value)
{
  struct Node* newNode = (struct Node*)
      *malloc (size of (struct Node));
  if (new Node == NULL) {
    Pf ("Memory allocation failed\n");
    return;
  }
  newNode -> data = value;
  newNode -> next = NULL;
//Handle 1st ele. insertion.
if (front == NULL && rear == NULL)
{
  front = rear = newnode;
}
else {
  rear -> next = newNode;
  rear = New Node;
}
```




```
  Pf ("%d enqueued to queue\n", value);
} //Function to dequeue
int dequeue () {
  if (front == NULL && rear == NULL)
  {return -1;
  }
  Struct Node* temp = front;
  int value = temp -> data;
  front = front -> next;
//Handle dequeuing the last ele
  if (front == NULL)
  {
    rear = NULL;
  }
  free (temp);
  return value;
}
int peek () {
  if (front == NULL)
  {
    return -1;
  }
  return front -> data;
}
void display ()
{
  if (front == NULL && rear == NULL)
  {
    Printf ("Queue is empty\n");
    return;
  }
  Struct Node* temp = front;
  Printf ("Queue elements : ");
  While (temp ! = NULL)
  {
    Printf ("%d", temp -> data);
    temp = temp -> next;
  }
  Printf ("\n");
```

Circular Linked List:-

- A circular linked list is a type of linked list in which the first and the last node are also connected to each other to form a circle. There are 2 types of circular linked list
  1. Circular Single linked list
  2. Circular Doubly linked list
- In Circular SLL the adress of last node consists of adress of 1st node.



Operations

(1) Insertion → at beg, → at End, → at Position.

(2) Deletion → at beg, → at end, → at Pos.

(3) Display

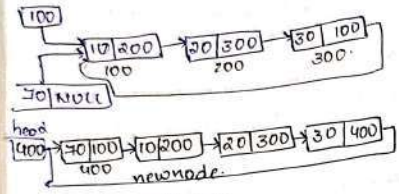
(4) Create.

Create:-

```
if (head = NULL)
{
  head = newnode
newnode -> next = head
}
Else {
  while (temp -> nex ! = hea
```

```
{ temp = temp -> next ;
  temp -> next = newnode ;
  newnode -> next = head ;
}.
```

Insert at beg:-




```
temp = head ;
newnode -> next = head ;
head = newnode ;
while (temp -> next -> next != head)
{
  temp = temp -> next ;
}
temp -> next = newnode ;
```

Insert at End:-

```
temp = head ;
while (temp -> next != head)
{
  temp = temp -> next ;
}
temp -> next = newnode ;
newnode -> next = head ;
```

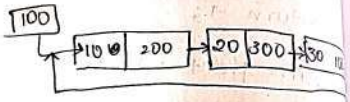
Insert at Position:-

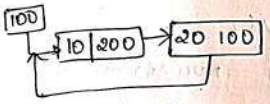
```
temp = head ;
for (int i=0; i<pos-1; i++)
{
  temp = temp -> next ;
  newnode -> next = temp -> next ;
  temp -> next = newnode .
}
```

Delete at beginning:-

```
temp = head ; d = head .
while (temp -> next != head)
  temp = temp -> next
}
head = d -> next ;
d -> next = NULL ;
free(d)
```

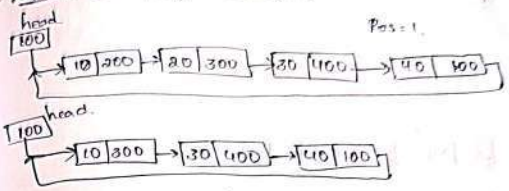
* Delete at End:-






```
temp = head
while (temp -> next -> next != h
{
  temp = temp -> next ;
}
d = temp -> next ;
temp -> next = head ;
d -> next = NULL ;
free (d)
```

* Delete at Specific Position:-

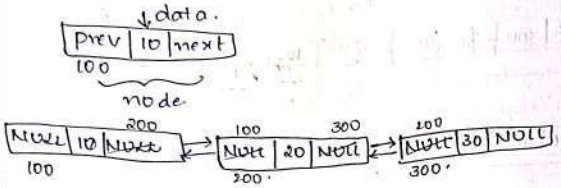



```
temp = head
for (i=0; i<pos-1; i++)
  temp = temp -> next ;
  d = temp -> next ;
  temp -> next = d -> next ;
  d -> next = NULL ;
  free (d) ;
```

* Display()

```
temp = head ;
while (temp -> next != head) {
  Printf ("%d", temp -> data ) ;
  temp = temp -> next ;
}
Printf ("%d" temp -> data) ;
```

Double Linked List:-



⇒ Struct node
{
struct node* prev;
int data;
struct node* next;
};

Functions of DLL:-

1. Create
2. Display
3. Insertion — at beg / at End / at pos.
4. Deletion — at beg / at End / at pos.

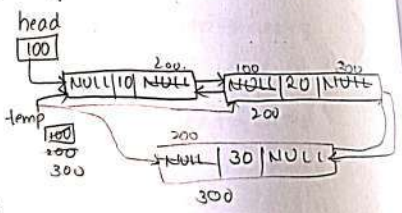
About DLL

- Double List navigation is possible in two ways that is forward and backward direction.
- In DLL node consists of 3 parts.
  1. Data part
  2. Pointer to next node.
  3. Pointer to the previous node.

Create:-

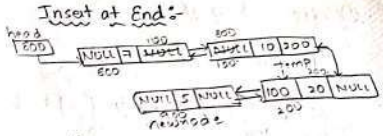
newnode = (struct node*) malloc (size of (struct node));

newnode → data = value;
newnode → prev = NULL;
newnode → next = NULL;
if (head == NULL)
{
temp = head = newnode;
}



Else
{
temp → next = newnode;
newnode → prev = temp;
temp = newnode;
}

Insert at End:-



{
temp = head;
While (temp → next != NULL)
{
temp = temp → next;
}
temp → next = newnode;
newnode → prev = temp;
}

~~Insert at beg:-~~

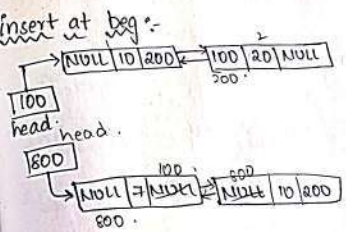
display():-

/ (temp != NULL)
{
while (temp → next != NULL)
{
~~temp~~ Pf ("%d", temp → data);
temp = temp → next;
}
Pf ("%d", temp → data); (or)
(No need to write this) for temp != NULL)
}

Insert at Pos:- pos(2)

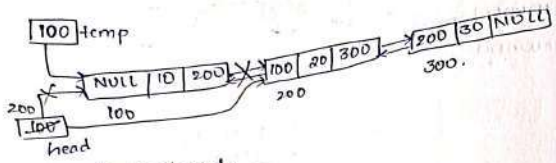
int i, pos;
struct node* temp = head
for (i=0; i<pos-1; i++) {
temp = temp → next;
}
temp → next = newnode;
newnode → prev = temp;
newnode → next = temp → next;
temp → next = newnode;
newnode → prev = temp;
temp → next → prev = newnode

insert at beg:-



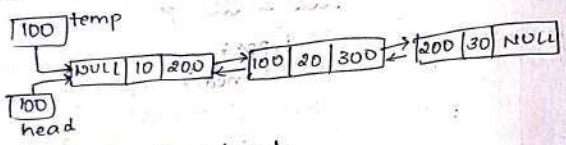
new node → next = head.
head → prev = newnode.
head = newnode.

Deletion at beg():-




```
temp = head;
head = head -> next;
temp -> next = NULL;
free(temp);
```

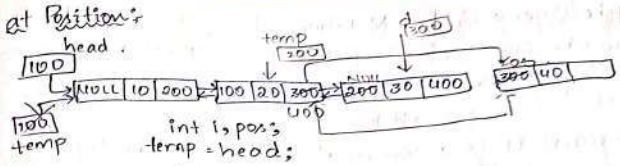
Delete at End:-




```
temp = head;
while (temp->next->next != NULL)
{
    temp = temp -> next;
}
d = temp->next;
temp->next = NULL;
d -> prev = NULL;
free(d);
```

another pointer

Delete at Position:-




```
int i, pos;
temp = head;
for(i=0; i<pos-1; i++)
{ temp = temp -> next;
  d = temp -> next;
  temp -> next = d -> next;
  d -> next -> prev = temp;
  d -> prev = NULL;
  d -> next = NULL;
  free(d);
```

Applications of Stack:-

1) Infix to postfix Conversition.
2) Postfix Evaluation.
3) Balancing Parenthesis.

"Infix to Postfix Conversition"

Infix Notation:- The expression which is in normal format i.e., the operator in between the operands.

Ex:- <operand> operator <operand>

a + b → operator.

operands

Prefix | Polishe Notation:- The expression which have operator comes first followed by operands.

<operator <operand> <operand>

Ex:- | +ab |

Postfix / Reverse Polishd Notation:- The expression which has operands comes first followed by operator.

<operand> <operand> operator

Ex: ab +

Expression:- It is a combination of operators & operand.

Ex: a+b* c

Precedence of an Operators:-

- Priority of the operators are predefined & on
- If any expression contains more than 1 operator which operator evaluate first indicated by precidence.
- We will conside 5 different operators those are

'+', '-', '*', '/', '%', '^'

table

			Rule.
1.	( ), { }, [ ],	← 1st priority	
2.	^	← 2nd "	Rule
3.	*, /, %	← 3rd priority	left to right
4.	+, -	← 4th Priority	left to right.

Rules for Conversion From Infix to Postfix expression

- Scan the expression character by character from left to right
- Print the operand as they arrive
- If the stack is empty (or) Contains a left paranthesis on top, push the incoming operator onto the stack.
- If the incoming Symbol is "(" push it on the stack
- If the incoming Symbol is ")" pop the stack & Print the operators until the left paranthesis is found.
- If the incoming Symbol has higher precidence then the top of the stack, push it onto the stack.
- If the incoming Symbol has lower/equal precidence then the top of the stack, Pop and Print the top of the stack. then text the incoming operator against the new top of the stack.
- At the end of the expression formed and. print all the operators of the stack.

eg:- (a+b/c * (d+e)-f)

Symbol	Stack	Postfix Expression.
(	(	
A	(	a
+	(+	a
b	(+	ab
/	(+/	ab
c	(+/ Because of highest Priority so not be pushed some	abc
*	(+(*) pop	abc/
	(+*	abc/
(	(+*(	abc/
d	(+*(	abc/d
+	(+*(+	abc/d
e	(+*(+	abc/de

)	(+*(+) pop	abc/de+
	(+*	
-	(+* pop	abc/de+*
	(+ pop	abc/de+*+
	(-	
F	(-	abc/de+*f
)	(-) pop	

abc/de+*f-

Final postfix expression

Eg2:- A+(B*C-(D/E^F)*G)*H. ← Infix expression.

Symbol	Stack	Postfix
A		A
+	+	A
(	+(	AB
B	+(	AB
*	+(*	ABC
C	+(*	ABC*
-	+(-) pop	
(	+(-	ABC*.
(	+(-(	ABC*D
D	+(-(	ABC*D
/	+(-(/	ABC*DE
E	+(-(/	ABC*DE
^	+(-(/^	ABC*DEF
F	+(-(/^	
)	+(-(/^) pop	ABC*DEF^
	+(-(/) pop	ABC*DEF^/
	+(-( )	ABC*DEF^/
*	+(-(*	ABC*DEF^/
G	+(-*	ABC*DEF^/G
)	+(-*) pop	ABC*DEF^/G*
	+(-) pop	ABC*DEF^/G*-
*	+*	ABC*DEF^/G*-
H	+(*) pop	ABC*DEF^/G*-H

Lastly. Remaining all will pop.

$\boxed{ABC*DEF^/G*-H*+}$ Final Postfix.

Eg:- (A+B)*(C-D)

Symbol	Stack	Postfix
(	(	
A	(	A
+	(+	A
B	(+	AB
)	(+) pop	AB+
*	*	AB+
(	*(	AB+
C	*(	AB+C
-	*(-	AB+C
D	*(-	AB+CD
)	*(-) pop	AB+CD-
	*	

$\boxed{AB+CD-*}$

Eg:- A+b*c-d/e*f.

Symbol	Stack	Postfix
A		A
+	+	A
b	+	Ab
*	+*	Ab
c	+* pop	Abc
-	+ pop	Abc*+
d	-	Abc*+d
/	-/	Abc*+d
e	-/	Abc*+de
*	-/*	Abc*+de
f	-/*	Abc*+de

$\boxed{Abc*+de*/-}$ Final.

Double Linked List (Program)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

struct Node* head = NULL;

struct Node* createNode(int data){
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node
    if (newNode == NULL) {
        Printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
=> void insertAtBeginning(int data){
    structNode *newNode = CreateNode(data);
    if (head == NULL) {
        head = newNode;
    }
    else {
        newNode->next = head;
        head->prev = newNode;
        head = new Node;
    }
}
```

```
=> void insertAtMiddle
        (int data, int position)
    if (position < 0) {
        Pf("Invalid position\n");
        return;
    }
    struct Node *newNode = CreateNode
        (data);
    if (head == NULL) {
        head = newnode;
        return;
    }
    struct Node *temp = head;
    int i;
    for (i=0; i<pos-1 && temp->next
        NULL ; i++) {
        if (temp->next != NULL) {
            temp = temp->next;
        }
    }newnode->next = temp->next;
    temp->next->prev = newNode;
    temp->next = newNode;
    newNode->prev = temp;
}
=> void insertAtEnd(int data) {
    struct Node *newNode = CreateNode
        (data);
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}
=> void deleteAtbeginning() {
    if (head == NULL) {
        Printf("List is Empty\n");
        return;
    }
    struct Node * temp = head;
    head = temp->next;
    if (head != NULL) {
        head->prev = NULL;
    }
    free(temp);
}
```

```
-> void deleteAtMiddle(int position){
    if (head == NULL){
        Printf("List is Empty\n");
        return;
    }
    if (position<0){
        Printf("Invalid position\n");
        return;
    }
    struct Node* temp = head;
    struct Node *temp2;
    int i;
    for(i=0; i<position && temp != NULL; i++){
        temp = temp->next;
    }
    if (temp == NULL){
        Printf("Invalid position\n");
        return;
    }
    if (temp->prev != NULL) {
        temp2 = temp->prev;
        temp2->next = temp->next;
        temp2->next->prev = temp2;
    }free(temp);
    temp = NULL;
}
=> void deleteAtEnd() {
    if (head == NULL) {
        Printf("List is Empty\n");
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL){
        temp = temp->next;
    }
    if (temp->prev != NULL) {
        temp->prev->next = NULL;
    }
```

```
    else {
        head = NULL;
    }
    free(temp);

→ void displayList() {
    Printf ("Double Linked list:");
    struct Node *temp = head;
    while (temp != NULL) {
        Printf ("%d", temp->data);
        temp = temp->next;
    }
    Printf ("\n");
}

→ int main() {
    int choice, data, position;
    do {
        Printf ("In 1. Insert at Beginning\n");
        Printf ("2. Insert at Middle\n");
        Pf("3. Insert at End");
        Pf("4. Delete at Beginning\n");
        Pf("5. Delete at Middle\n");
        Pf("6. Delete at End\n");
        Pf("7. Display list\n");
        Pf("8. Exit\n");
        Printf ("Enter your choice: ");
        scanf ("%d", &choice);
switch(choice){
    case 1:
        Pf("Enter data to insert at Beg: ");
        scanf("%d", &data);
        insertAt Beginning (data);
        break;
    case 2:
        Pf("Enter data to insert: ");
        scanf("%d", &data);
        Pf("Enter pos. to insert at: ");
        scanf("%d", &position);
        insertAt Middle (data, position);
        break;
    Case 3:
        Pf("Enter data to insert at End...
        scanf("%d", &data);
        insert at End (data);
        break;
    Case 4:
        deleteAt Beginning ();
        break;
    Case 5:-
        Printf ("Enter pos. to delete: ");
        scanf("%d", &position);
        delete At Middle (position);
        break;
    Case 6:-
        delete At End ();
        break;
    Case 7:-
        displayList ();
        break;
    Case 8:-
        Printf("Exiting Program.\n");
        break;
    default:
        Pf("Invalid choice. Please try
            again. \n");
    }
    while (choice != 8);
    return 0;
```

Infix to Postfix

Eg:- a+(b*c-d)/e.

Symbol	stack	Postfix expression.
a		a
+	+	
(	+(	
b	+(	ab
*	+(*	ab
c	+(*	ab
-	+(-	abc
d	+(-	abc*
)	+(-) pop	abc*d
/	+/	abc*d-
e	+/	abc*d-
		abe*d-

Final expression.

$\boxed{abc*d-e/+}$

Code:-

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
char stack [100];
int top = -1;
void push (char x) {
    top = top+1;
    stack [top] = x;
}
char pop() {
    if (top == -1)
        return -1;
    else
        return stack[top--];
}

int priority(char x)
{
    if (x == '(')
        return 0;
    if (x == '+' || x == '-')
        return 1;
    if (x == '*' || x == '/')
        return 2;
}
int main() {
    char exp[100];
    char x;
    int len, i;
    Printf("Enter the expressio
    scanf ("%s", exp);
    Printf ("\n");
```

```
len = strlen (exp);
for (i = 0; i < len; i++)
{
  if (isalnum(exp[i]))
      Printf ("%c", exp[i]);
  else if (exp[i] == '(')
      push (exp[i]);
  else if (exp[i] == ')')
  {
    while ((x = pop()) != '(')
        Printf ("%c", x);
  }
  else {
    while (priority (stack [top]) >= priority (exp [i])) {
        Printf ("%c", pop ());
    }
    Push (exp[i]);
  }
}
while (top != -1) {
    Printf ("%c", pop());
}
return 0;
}
```

alphabet / number

Postfix Evaluation:

- In postfix expression it can be evaluated without parenthesis and it can be evaluated as 2 operands and an operator at a time and time becomes easier for the compiler and the computer to handle.

Evaluation Rules of Postfix Expression.

- When reading the expression from left to right push the element into the stack if it is an operand.
- Pop the 2 operands from the stack if the element is an operator and then evaluate it.
- Push back the result of the evaluation repeat it till the end of the expression.
- When the expression is ended the number in the stack is final answer.

Eg:- 4 5 6 * +

Reading Expression	Stacktop	Result
4	4	
5	5, 4	
6	6, 5, 4	
*	6, 5, 4 } pop	6*5=30 → Push into stack.
+	30, 4 } pop	4+30=34 Final answer.

Eg 2:- 231 * + 9 -

Reading Expression	Stack top	Result.
2	2	
3	3, 2	
1	1, 3, 2	
*	1, 3, 2 } pop	3*1 = 3 ✓ push to stack.
+	3, 2 } pop	3+2 = 5 ✓ push to stack
9	5, 9	
−	5, 9 } pop	9-5 = 4

③ 53 + 62 /* 35 *+

Reading Expression	Stack	Result
5	5	
3	3, 5	
+	3, 5 } pop	5+3 = 8 ✓, push to stack
6	6, 8	
2	2, 6, 8	
/	2, 6, 8 } pop	6/2 = 3 push.
*	3, 8 } pop	8*3 = 24
3	3, 24	
5	5, 3, 24	
*	5, 3, 24 } pop	3*5 = 15
+	15, 24 } pop	24+15 = 39.

Eg 4:- ~~AB +~~ ~~AB*~~ (A*B) 3 5 5 5 /* + 2 2 ^ − → (4)

Eg-5:- 8 5 4 ^ 2 + * 6 2 ^ 9 3 /− (5014.67)

Reading expression	Stack	Result.
3	3	
5	5, 3	
5	5, 5, 3	
5	5, 5, 5, 3	
/	5, 5, 5, 3 } pop.	5/5 = 1
*	1, 5, 3 } pop	5*1 = 5
+	5, 3 } pop	3+5 = 8
2	2, 8	
2	2, 2, 8 } pop	
^	2, 2, 8 } pop	2^2 = 4
−	4, 8 } pop	8-4 = 4

Program:-

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int stack[20];
int top = -1;

void push (int x){
    top = top+1;
    stack[top] = x;
}
int pop() {
    return stack[top--];
}
int main() {
    char exp[20];
    int len, n1, n2, n3, num, i;
    Printf("Enter the expression: ");
    scanf("%s", exp);
    len = strlen(exp);
    for(i=0; i<len; i++) {
        if (isdigit(exp[i])) {
            num = exp[i] - 48;
            push(num);
        }
        else {
            n1 = pop();
            n2 = pop();
            switch (exp[i]) {
                case '+':
                {
                    n3 = n2 + n1;
                    break;
                }
```

asci value

0 — 48
1 — 49
2 — 50
3 — 51
4 — 52
5 — 53
6 — 54
7 — 55
8 — 56
9 — 57

```
                case '-':
                {
                    n3 = n2 - n1;
                    break;
                }
                case '*':
                {
                    n3 = n1 * n2;
                    break;
                }
                case '/':
                {
                    n3 = n2/n1;
                    break;
                }
                push(n3);
            }
        }
    printf("\n The result of expression %s = %d\n\n", exp, pop());
    return 0;
}
```

Balancing Paranthesis

- The parathesis is represented by the brackets shown below.

( ) where:- (— opening brackets these paranthesis are used to represent
) — closing brackets.

the mathematical representation. The balanced paranthesis means that when the opening paranthesis is equal to the closing paranthesis.

Balanced	Unbalanced.
(a+b)	{a-b)
[(a+b)-c]	{(a+b) - c/f]}
[{(a-b)/(d+e)}]	[(a-b+e-f/d})]

Rules:-

- Traversal string
- If the incoming character is starting bracket '{' & open bracket '(', '[' push into the stack.
- If the incoming character is closing bracket '}', ')', ']' and top of the stack is starting bracket then pop from the stack.

At the end of the string the stack is empty then it is balanced otherwise it is not balanced.

Program:-

```
#include <stdio.h>
#include <string.h>
char stack [100];
int top = -1;
void push (char x) {
    stack [++top] = x; //Increment top & then assign
}
void pop() {
    if (top >= 0) //Ensure stack is not empty before popping.
        top--;
}
int main () {
    char exp[50], ch;
    int i, len;
    Printf ("Enter expression: ");
    Scanf ("%s", exp);
    len = strlen (exp);
    for (i=0; i<len; i++) {
        ch = exp[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            Push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (top == -1) {
                Printf ("Not Balanced\n");
                return 0;
            }
            if ((ch == ')' && stack [top] == '(') || (ch == ']' &&
                stack [top] == '[') || (ch == '}' && stack [top] =
```

```
        {
          pop();
        }
        else {
          Printf("Not balanced \n");
          return 0;
        }
    }
}
  if (top==-1)
     Printf("Balanced \n");
  else
     Printf("Not Balanced \n");
  return 0;
}
```

16 15-02-25

Types of Queues:-

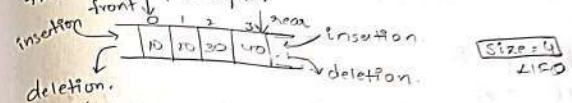
3 types of Queues

1. Circular Queue (using Arrays)
2. Dequeue (Double ended queue)
3. Priority Queue (Using Arrays)

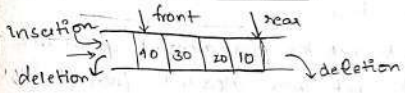
"Dequeue:-"

- In dequeue insertion and ~~car~~ deletion can be done from both end of the queue either from front end or rear end.
- It means that we can insert and delete elements from both front and rear ends of the Queue.
- Dequeue can be used as a palindrome check means that if we read the string from both end then the string would be same.
- Dequeue can be used both as stack and Queue as it allows the insertion and deletion operation on both ends.

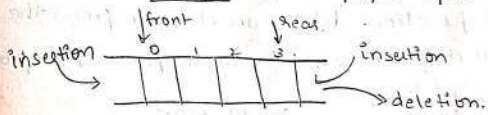
- Dequeue can be considered as stack because stack follows the LIFO rule i.e., last in first out principle in which insertion and deletion can be performed only from one end. and in dequeue it is possible to perform both insertion and deletion from one end. and dequeue doenot follow the FIFO principle.



- There are 2 types of dequeue. 1) Input Restricted dequeue:- as a name implies in input Restricted dequeue insertion operation can be perform at only one end while deletion can be perform as the both ends.



2) Output Restricted dequeue:- As the name implies in output Restricted Queue deletion operation can be performed at only one end while insertion can be perform from bothends

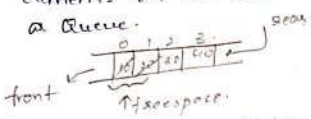


Operations of dequeue:-

1) Insert at rear end
2) Delete at front end
3) Insert at front end
4) Delete at rear end

"Circular Queue":

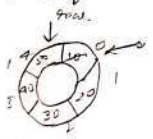
Drawback of General Queue:- In general Queue we have the limitation i.e., Once the Queue is full eventhough elements from front are deleted and there is a freespace but it is not possible to add elements in the Queue so we move to the concept call circular a Queue.



- A circular Queue is similar to linear Queue as it is also based on FIFO rule i.e., first in first out principle except that the last position is connected to the first position in a circular Queue that forms a circle is also called as Ring buffer.

Operation of Circular Queue.

1) Front:- It is used to get the front elements from the queue.
2) Rear:- It is used to get rear elements from the queue.
3) Enqueue:- This function is used to insert a new value in the Queue. The new element is always inserted from the rear end.
4) Dequeue:- The function deletes an element from the queue. the deletion in a queue always takes from the front end.



Initial pos:-
front = -1
rear = -1

$rt = (front+1)\%N$; $rear = (rear+1)\%N$ → size of the Queue

$= (-1+1)\%5$; $= (-1+1)\%5$

$= 0\%5 = 0$; $= 0\%5 = 0$

"Applications":-

1. Circular Memory Management: The Circular Queue Provides Memory mangement as in linear queue it is not managed very efficiently. but in case of circular Queue the memory is managed efficiently by placing the elements in a location which is unused.
2. CPU scheduling: The operating system also uses the circular Queue to insert the process and execute them.
3. Traffic System:- Traffic light is one of the best example of the circular Queue each light of traffic light gets on one by one after every interval of time.

Program:-

```
#include <stdio.h>
#include <stdlib.h>

#define N 5

int queue[N];
int front = -1;
int rear = -1;

void enqueue(int element);
void dequeue();
void display();
int main() {
  int ch, element;
  while(1) {
      Printf("\nmenu\n 1.enque \n 2.dequeue\n 3. display\n4
          exit\n");
```

```
Scanf("%d",&ch);
switch(ch) {
    case 1:
        scanf("%d",&element);
        enqueue(element);
        break;
    case 2:
        dequeue();
        break;
    case 3:
        display();
        break;
    case 4: exit(0);
        break;
    default:
        Printf("enter a right choice");
    }
}
void enqueue(int element) {
    if (front==-1 && rear==-1) {
        front=rear=0;
        queue[rear]=element;
        Printf("insertion is success");
    }
    else if ((rear+1)%N==front) {
        Printf("queue is full");
    }
    else {
        rear=(rear+1)%N;
        queue[rear]=element;
        Printf("insertion is success");
    }
}
```

```
void dequeue() {
    if (front==-1 && rear==-1) {
        Printf("queue is empty");
    }
    else if (front==rear) {
        front=-1;
        rear=-1;
        Printf("deletion is successful");
    }
    else {
        front=(front+1)%N;
        Printf("deletion is success");
    }
}
void display() {
    int i=front;
    if (front==-1 && rear==-1) {
        Printf("queue is empty");
    }
    else {
        while (i!=rear) {
            Printf("%d", queue[i]);
            i=(i+1)%N;
        }
        Printf("%d",queue[i]);
    }
}
```